



Valor 5%

Tema: Ingeniería Inversa

Modalidad: GRUPAL

Entregable: Video con la explicación de los pasos seguidos donde participen todos y la cámara esté encendida mientras ejecutan los pasos del laboratorio.

En caso que deban ajustar alguna instrucción para hacer funcionar el laboratorio deben indicarlo en el video.

Laboratorio práctico: Ingeniería inversa (desensamblado, decompilado y debugging)

Objetivo general: Comprender, de forma práctica, cómo recuperar la lógica de un programa compilado usando análisis estático (strings, desensamblado, decompilado) y análisis dinámico (debugging).

1) Requisitos y entorno seguro

- **Sistema:** Linux (Ubuntu/Kali) recomendado en **máquina virtual**. También funciona en macOS; Windows puede usarse con **WSL** o **MinGW-w64**.
- **Herramientas:**
 - Compilador C (gcc o clang) y terminal.
 - strings, file, sha256sum, objdump (o otool en macOS).
 - **Ghidra** (desensamblador y decompilador con GUI).
 - **GDB** (debugger). En macOS, puede usarse **LLDB**.
- **Archivos del laboratorio:** safecheck.c, obfcheck.c, Makefile, build.sh (se entregan en el paquete ZIP).



Ética y legalidad: Este laboratorio usa programas benignos creados para fines educativos. No apliques ingeniería inversa a software propietario sin permiso.

2) Objetivos específicos

1. Reconocer metadatos y cadenas útiles en un binario.
2. Desensamblar y decompilar para ubicar la lógica de validación.
3. Usar el depurador para observar argumentos de funciones en tiempo de ejecución.
4. (Extra) Alterar el flujo en runtime para comprender el impacto de parches/bypasses.

3) Preparación

1. Descarga y descomprime el paquete del laboratorio.
2. En Linux/macOS, abre una terminal en la carpeta del lab.
3. Compila con el script:
4. `chmod +x build.sh`

`./build.sh`

Esto creará `bin/safecheck`, `bin/safecheck_stripped` y `bin/obfcheck`.

Si prefieres comandos manuales:

```
gcc -g -O0 src/safecheck.c -o bin/safecheck
```

```
cp bin/safecheck bin/safecheck_stripped && strip bin/safecheck_stripped
```

```
gcc -g -O0 src/obfcheck.c -o bin/obfcheck
```

4) Parte A – Reconocimiento estático rápido

Trabajaremos sobre `bin/safecheck` primero.

1. Tipo de archivo y arquitectura



file bin/safecheck

Identifica formato (ELF Mach-O/PE), arquitectura (x86_64, ARM) y si es dinámico/estático.

2. Hash

sha256sum bin/safecheck

Útil para integridad y referenciar la muestra.

3. Cadenas visibles

strings -n 4 bin/safecheck | less

Busca mensajes como "Ingrese la clave", "Acceso concedido".

Conclusión: a veces ya hay pistas sin ejecutar el binario.

4. Símbolos/Secciones (si no está stripped)

objdump -t bin/safecheck | head

Si ves nombres como main, el binario **no** está “stripped”. Compara con bin/safecheck_stripped.

macOS: usa otool -L para librerías y nm para símbolos.

5) Parte B – Desensamblado y decompilado con Ghidra

1. **Abrir el proyecto** en Ghidra → *File* → *New Project* → *Non-Shared Project*.
2. *File* → *Import File...* → selecciona bin/safecheck → *OK*.
3. Doble clic en el binario → *Yes* para **Analyze** → deja opciones por defecto → *Analyze*.
4. En la **Symbol Tree**, abre main.
5. Vista **Decompile** (panel derecho): identifica la lógica principal.
 - Observa la llamada a scanf y la comparación con strcmp.



- Debe aparecer una constante clara (p. ej., "Cyber2025").
- 6. **Etiquetado y notas:** renombra funciones/variables (R-click → *Rename*) y añade comentarios.
- 7. **Graph de funciones:** visualiza el flujo entre funciones (*Window* → *Function Graph*).

Repite el proceso con bin/obfcheck: verás una función que **de-ofusca** la clave restando 1 a cada carácter antes de comparar.

6) Parte C – Debugging (dinámico) con GDB

Objetivo: interceptar la llamada a strcmp y observar sus argumentos.

1. Lanza GDB:

```
gdb -q bin/safecheck
```

2. Inicia el programa y detente al comienzo de main:

```
start
```

3. Coloca un breakpoint en strcmp y continúa:

4. `break strcmp`

```
continue
```

5. Cuando GDB se detenga en strcmp, inspecciona **argumentos** (x86_64 SysV):

- Primer argumento (RDI) → entrada del usuario.
- Segundo argumento (RSI) → cadena objetivo.

6. `x/s $rdi`

```
x/s $rsi
```

Esperado: verás tu input y la clave real en memoria.

7. Deja que strcmp retorne a main y **fuerza el resultado** (bypass educativo):



8. finish # vuelve a la llamada en el caller

9. set \$rax=0 # finge que strcmp devolvió 0 (iguales)

continue

Modifica el flujo sin tocar el binario en disco y observa lo que sucede

Tip: en ARM64/M1 el registro de retorno es x0. En macOS con LLDB, comandos análogos: breakpoint set --name strcmp, register read, thread return -x 0.

8) Cierre y recordatorio ético

Este laboratorio demuestra cómo combinar análisis estático y dinámico para comprender la lógica de un binario. Úsalo para **defensa, auditoría y aprendizaje**, respetando licencias y marco legal vigente.